MODULE 5

Object Oriented Programming

Topics Covered

Lecture 1 — OOP Introduction
Lecture 2 — OOP Concepts (Classes, Objects, Methods)
Lecture 3 — Inheritance
Lecture 4 — Polymorphism
Lecture 5 — Encapsulation
Lecture 6 — Abstraction
Lecture 7 — Magic Methods
Lecture 8 — Operator Overloading

Lecture 1: OOP Introduction

OOP organises code around objects — entities that bundle data (attributes) and behaviour (methods). Instead of a sequence of instructions, you model the world as interacting objects.

> **Analogy**
> A Car class is a blueprint. Every physical car is an object (instance). Each car has the same structure but its own values — one is red, another blue.

Why OOP?

- Modularity — code split into self-contained classes
- Reusability — classes extended via inheritance
- Maintainability — changes in one class don't break everything
- Industry standard — Django, TensorFlow, PyTorch are all OOP

Four Pillars at a Glance

Pillar	One-line Definition
Encapsulation	Bundling data + methods; hiding internals
Abstraction	Exposing only what's necessary
Inheritance	A class reusing/extending another class
Polymorphism	Same interface, different behaviour

Lecture 2: OOP Concepts

Class, Object & __init__

A class is a blueprint. An object is a concrete instance. __init__ is the constructor — called automatically when an object is created.

```
class Dog:
    species = 'Canis familiaris'      # class attribute (shared)

    def __init__(self, name, age):    # constructor
        self.name = name              # instance attribute (unique)
        self.age  = age

    def bark(self):
        print(f'{self.name} says: Woof!')

d1 = Dog('Buddy', 3)
d2 = Dog('Max', 5)
d1.bark()           # Buddy says: Woof!
print(Dog.species)  # Canis familiaris
```

self

self refers to the current instance. Python passes it automatically — you never pass it manually when calling a method.

Instance vs Class vs Static Methods

Type	First Param | When to Use
Instance method	self — access/modify instance data (most common)
Class method	cls — access class-level data; @classmethod
Static method	none — utility; no access to instance or class; @staticmethod

Lecture 3: Inheritance

Inheritance lets a child class acquire attributes and methods of a parent class — establishing an IS-A relationship. Code is written once and reused.

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        print(f'{self.name} makes a sound.')

class Dog(Animal):              # Dog IS-A Animal
    def __init__(self, name):
        super().__init__(name)  # call parent constructor
    def speak(self):            # override parent method
        print(f'{self.name} says Woof!')
    def fetch(self):
        print(f'{self.name} fetches!')

d = Dog('Buddy')
d.speak()   # Buddy says Woof!  (overridden)
d.fetch()   # Buddy fetches!    (own method)
```

Types of Inheritance

Type	Description
Single	One child, one parent: Dog(Animal)
Multiple	One child, multiple parents: Duck(Flyable, Swimmable)
Multilevel	Chain: Dog → Mammal → Animal
Hierarchical	Multiple children from one parent: Dog(Animal), Cat(Animal)

MRO — Method Resolution Order

Python uses C3 linearisation to decide which parent method runs in multiple inheritance. Check with: print(ClassName.__mro__)

Lecture 4: Polymorphism

Polymorphism means 'many forms' — the same method name behaves differently depending on the object. Enables writing generic code that works with different types.

```
class Cat:
    def sound(self): return 'Meow!'

class Dog:
    def sound(self): return 'Woof!'

class Robot:
    def sound(self): return 'Beep!'

# No common parent needed — Duck Typing!
for obj in [Cat(), Dog(), Robot()]:
    print(obj.sound())    # Meow!  Woof!  Beep!
```

Duck Typing
If it has the method you call, it works — regardless of class. 'If it walks like a duck and quacks like a duck, it is a duck.'

Types of Polymorphism

Type	How
Method Overriding	Child redefines a parent's method (see Lecture 3)
Duck Typing	Any object with the right method works
Operator Overloading	Custom +, -, == etc. for your classes (Lecture 8)

Lecture 5: Encapsulation

Encapsulation bundles data and methods in a class while restricting direct access to internal data. Controls how attributes are read and modified.

Access Modifiers

Modifier	Syntax | Meaning
Public	self.name — accessible anywhere (default)
Protected	self._name — convention: internal use only
Private	self.__name — name-mangled; strongly discouraged externally

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner     = owner          # public
        self.__balance = balance        # private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

Modifier	Syntax | Meaning

```
    def get_balance(self):              # controlled getter
        return self.__balance

acc = BankAccount('Alice', 5000)
acc.deposit(1000)
print(acc.get_balance())   # 6000
# acc.__balance → AttributeError
```

@property — Pythonic Getters & Setters

```
class Temperature:
    def __init__(self, celsius):
        self.__c = celsius

    @property
    def celsius(self):           # getter
        return self.__c

    @celsius.setter
    def celsius(self, v):        # setter with validation
        if v < -273.15: raise ValueError('Below absolute zero!')
        self.__c = v

    @property
    def fahrenheit(self):        # computed, read-only
        return self.__c * 9/5 + 32

t = Temperature(25)
print(t.fahrenheit)   # 77.0
t.celsius = 100
print(t.fahrenheit)   # 212.0
```

Lecture 6: Abstraction

Abstraction hides complex implementation details and exposes only the essential interface. An abstract class acts as a contract — subclasses must implement marked methods.

```
from abc import ABC, abstractmethod

class Shape(ABC):              # cannot be instantiated
    @abstractmethod
    def area(self): pass       # subclasses MUST implement

    @abstractmethod
    def perimeter(self): pass

    def describe(self):        # concrete method
        print(f'Area={self.area():.2f}, Perimeter={self.perimeter():.2f}')

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self):        return 3.14159 * self.r ** 2
    def perimeter(self):   return 2 * 3.14159 * self.r
```

```
Circle(5).describe()   # Area=78.54, Perimeter=31.42
# Shape()  → TypeError: Can't instantiate abstract class
```

Abstraction	Encapsulation
Hides complexity of implementation	Hides the data itself
WHAT an object does	HOW data is protected
Abstract classes / interfaces	Access modifiers & @property

Lecture 7: Magic Methods

Magic (dunder) methods start and end with __. Python calls them automatically in specific situations — printing, comparison, iteration, etc.

Method	Triggered By
__init__	Object creation
__str__	print(obj) or str(obj)
__repr__	repr(obj) — developer view
__len__	len(obj)
__getitem__	obj[key]
__contains__	'x' in obj
__iter__ / __next__	for x in obj
__eq__ / __lt__	obj == other / obj < other
__call__	obj() — call like a function

```
class Book:
    def __init__(self, title, author, pages):
        self.title  = title
        self.author = author
        self.pages  = pages

    def __str__(self):
        return f'"{self.title}" by {self.author}'

    def __repr__(self):
        return f'Book({self.title!r}, {self.author!r}, {self.pages})'

    def __len__(self):   return self.pages
    def __eq__(self, o): return self.pages == o.pages
    def __lt__(self, o): return self.pages < o.pages

b1 = Book('Clean Code', 'Martin', 431)
b2 = Book('Python 101', 'Driscoll', 301)
print(b1)           # "Clean Code" by Martin
print(len(b1))      # 431
print(b2 < b1)      # True
books = [b1, b2]
print(sorted(books))  # sorted by pages via __lt__
```

Lecture 8: Operator Overloading

Operator overloading defines custom behaviour for standard operators (+, -, *, ==, < etc.) on user-defined objects using magic methods.

Operator	Magic Method
+	__add__
-	__sub__
*	__mul__
/	__truediv__
==	__eq__
<	__lt__
+=	__iadd__
str left *	__rmul__

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):  return f'Vector({self.x}, {self.y})'
    def __add__(self, o): return Vector(self.x+o.x, self.y+o.y)
    def __sub__(self, o): return Vector(self.x-o.x, self.y-o.y)
    def __mul__(self, s): return Vector(self.x*s,   self.y*s)
    def __rmul__(self, s): return self.__mul__(s)  # 3 * v
    def __eq__(self, o):  return self.x==o.x and self.y==o.y
    def __abs__(self):    return (self.x**2 + self.y**2)**0.5

v1 = Vector(2, 3)
v2 = Vector(1, 4)
print(v1 + v2)   # Vector(3, 7)
print(v1 * 3)    # Vector(6, 9)
print(3 * v1)    # Vector(6, 9)  — uses __rmul__
print(abs(v1))   # 3.605...
print(v1 == v2)  # False
```

Real-world use
NumPy arrays, Pandas DataFrames, and PyTorch tensors all use operator overloading — that's why np.array([1,2]) + np.array([3,4]) gives array([4,6]).

End of Module 5 Notes — Generative AI Course